

Eine Datenbank, zwei Theken

Wie VereinsDeckel-Geräte — Android wie iPad — auf denselben Datenbestand zugreifen, ohne dass eine Buchung verloren geht, wenn das WLAN im Vereinsheim für zwei Minuten weg ist.

Gegenstand	Festlegung
Architektur	Offline-First mit Synchronisation. Das Gerät bleibt allein verkaufsfähig; der Server ist die gemeinsame Wahrheit.
Server	PostgreSQL 15+ und ein HTTP-Dienst. Sprache freigestellt.
Schnittstelle	REST über HTTPS, JSON, zwei Sync-Endpunkte plus Geräte-Anmeldung.
Clients	Android und iOS aus gemeinsamer Kotlin-Multiplatform-Basis.
Voraussetzung	Drei Änderungen am Datenmodell (Kapitel 2). Ohne sie ist Mehrgerätebetrieb nicht korrekt möglich.

Was dieses Dokument leistet

Es beschreibt Schema, Schnittstelle und Synchronisationsregeln vollständig genug, dass der Server unabhängig von der App gebaut werden kann. Die Kapitel 2 bis 4 sind die eigentliche Substanz — Kapitel 5 ist Referenz zum Nachschlagen.

	Inhalt
1	Ausgangslage und Grundsatzentscheidung
2	Die drei Brüche im heutigen Datenmodell — Schlüssel, Saldo, Bestand
3	Serverschema
4	Synchronisationsprotokoll
5	Schnittstellenreferenz
6	Kartenzahlung im Mehrgerätebetrieb
7	Betrieb
8	Abnahmekriterien

1 · Ausgangslage und Grundsatzentscheidung

Die App speichert heute alles lokal in SQLite über Room: elf Tabellen, Schema-Version 10, fortlaufende Ganzzahl-Schlüssel. Genau ein Gerät, genau ein Datenbestand. Sobald ein zweites Gerät dazukommt, brechen drei Annahmen dieses Entwurfs gleichzeitig.

1.1 Warum nicht einfach alles über den Server

Der naheliegende Weg wäre ein dünner Client: jede Abfrage und jede Buchung geht direkt an die API, der Server hält den Zustand. Das ist einfacher zu bauen und für eine Kasse trotzdem die falsche Wahl.

- Ein Vereinsheim-WLAN ist kein Rechenzentrum. Fällt es während des Sommerfests aus, steht eine Kasse, vor der eine Schlange steht.
- Der Verkaufsweg verträgt keine Netzlatenz. Ein Produkt-Tap muss sofort im Warenkorb liegen, nicht nach einem Roundtrip.
- Der Ausfall ist genau dann am wahrscheinlichsten, wenn der Betrieb am höchsten ist — viele Menschen, viele Geräte, ein überlasteter Accesspoint.

Entscheidung

Jedes Gerät behält seine vollständige lokale Datenbank und bleibt ohne Netz uneingeschränkt verkaufsfähig. Änderungen werden lokal geschrieben, in eine Warteschlange gelegt und bei Gelegenheit zum Server geschoben. Der Server ist die gemeinsame Wahrheit, nicht die Voraussetzung für den Betrieb.

1.2 Was der Server dadurch leisten muss

Offline-First verschiebt die Schwierigkeit von der Verfügbarkeit zur Korrektheit. Wenn zwei Kassen zwei Minuten lang unabhängig gebucht haben, muss der Server beide Historien zu einem Ergebnis zusammenführen, das ein Kassier nachvollziehen kann. Das geht nur, wenn von vornherein feststeht, welche Daten überhaupt kollidieren können — und die Antwort darauf ändert das Datenmodell.

2 · Die drei Brüche im heutigen Datenmodell

Diese drei Punkte sind keine Verbesserungsvorschläge. Ohne sie verliert der Mehrgerätebetrieb Geld, und zwar still.

2.1 Schlüssel: fortlaufende Zahlen kollidieren

Alle elf Tabellen nutzen `@PrimaryKey(autoGenerate = true)` auf einem Long. Legt die Theke ein Produkt an, bekommt es die 5. Legt das iPad zeitgleich ein anderes Produkt an, bekommt es ebenfalls die 5. Beim Zusammenführen sind das zwei verschiedene Dinge mit demselben Namen — der Server kann sie nicht unterscheiden, und eines überschreibt das andere.

Lösung: UUID als Primärschlüssel, erzeugt vom Client. Empfohlen wird UUIDv7: zeitsortiert, also anders als UUIDv4 indexfreundlich und in Listen natürlich chronologisch.

```
-- vorher (Client, Room)
id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT

-- nachher (Client und Server identisch)
id TEXT NOT NULL PRIMARY KEY           -- Client/SQLite: UUID als Text
id UUID NOT NULL PRIMARY KEY          -- Server/PostgreSQL
```

Die Migration vergibt für jede bestehende Zeile einmalig eine UUID und schreibt die Fremdschlüssel über eine Abbildungstabelle `alt->neu` um. Siehe 2.5.

2.2 Saldo: ein veränderlicher Zähler verliert Buchungen

`members.balance` wird heute additiv fortgeschrieben — im DAO steht wörtlich `UPDATE members SET balance = balance + :amount`. Auf einem Gerät ist das korrekt. Auf zweien nicht:

```
Deckel von M. Bauer: 20,00 EUR

10:15:02 Theke   liest 20,00 -> bucht Bier 4,00 -> schreibt 16,00
10:15:04 iPad    liest 20,00 -> bucht Wein 5,00 -> schreibt 15,00

Ergebnis: 15,00 EUR. Das Bier ist bezahlt worden, steht aber nirgends.
Der Verein hat 4,00 EUR verschenkt, und niemand kann es nachvollziehen.
```

Klassisches Lost Update. Es fällt nicht auf, weil das Ergebnis plausibel aussieht.

Lösung: Der Saldo wird nicht mehr gespeichert, sondern aus den Buchungen abgeleitet. Die Tabelle `transactions` enthält bereits jede Bewegung — Aufladungen wie Belastungen. Sie ist anfügend und kollidiert damit nie. Clients dürfen `balance` nicht mehr senden; der Server berechnet und liefert ihn.

Die Vorzeichenregel ergibt sich exakt aus dem heutigen Code (`AppRepository.adjustMemberBalance` und `SalesViewModel`):

```
CREATE OR REPLACE VIEW member_balances AS
SELECT m.id AS member_id,
       COALESCE(SUM(
         CASE
           -- Aufladung/Korrektur: Sentinel-Produkt, price traegt das Vorzeichen
           WHEN t.product_ref = '00000000-0000-0000-0000-000000000000'
             THEN t.price * t.quantity
           -- Verkauf auf den Deckel
           WHEN t.payment_type = 'MEMBER_BALANCE'
             THEN -(t.price * t.quantity - t.discount_amount)
           -- Bar und Karte beruehren den Deckel nicht
           ELSE 0
         END * CASE WHEN t.is_refund THEN -1 ELSE 1 END
       ), 0)::numeric(12,2) AS balance
FROM members m
LEFT JOIN transactions t
      ON t.member_id = m.id AND NOT t.deleted
GROUP BY m.id;
```

Der Sentinel ersetzt die heutige TOPUP_PRODUCT_ID = -1L. Bei großen Datenmengen wird daraus eine materialisierte Sicht, die per Trigger auf transactions aktualisiert wird.

2.3 Bestand: dieselbe Falle, nur im Keller

stock_items.simple_quantity, container_types.full_count und tapped_containers.drawn sind ebenfalls fortgeschriebene Zähler. Zwei Geräte, die gleichzeitig zapfen, verlieren nach demselben Muster Verbrauch — mit dem Unterschied, dass es hier keinen Kassenbon gibt, an dem es auffällt.

Lösung: Gleiches Prinzip. Der Bestand wird aus den anfügenden Tabellen hergeleitet: Zugänge aus stock_entries, Verbrauch aus transactions über die Rezeptur (product_components, skaliert mit der Variantengröße), Verderb aus tapped_containers.discarded_volume.

Hier ist Abgleich mit dem bestehenden Code nötig

Die Datei data/stock/Inventory.kt rechnet den verfügbaren Bestand und die gelernten Fasserträge heute schon weitgehend aus Ereignissen. Diese Logik wandert unverändert ins geteilte Modul und muss auf dem Server bitweise gleich abgebildet werden — sonst zeigen App und Server unterschiedliche Bestände an, was schlimmer ist als gar keine Serveranzeige.

2.4 Löschen: ein fehlender Datensatz ist keine Information

Wird eine Zeile hart gelöscht, kann der Server dem anderen Gerät nicht mitteilen, dass sie weg ist — Abwesenheit lässt sich nicht übertragen. Jede Tabelle bekommt daher deleted BOOLEAN und deleted_at; die App filtert gelöschte Zeilen in den Abfragen aus. Historientabellen (transactions, stock_entries) werden ohnehin nie gelöscht, sondern storniert.

2.5 Migrationspfad für bestehende Geräte

- **Schema 10 auf 11, lokal:** Neue Spalte uuid auf jeder Tabelle, gefüllt mit frisch erzeugten UUIDs.
- **Fremdschlüssel umschreiben:** Jede Referenzspalte wird über die alt-zu-neu-Abbildung auf die UUID gezogen, danach ersetzt die UUID-Spalte den Primärschlüssel.
- **Erstkontakt mit dem Server:** Das erste Gerät lädt seinen gesamten Bestand als Erstbefüllung hoch. Jedes weitere Gerät startet leer und zieht alles.

- **Kein Doppelbestand:** Genau ein Gerät wird beim Einrichten als Quelle markiert. Wird das versehentlich bei zweien gemacht, entstehen Dubletten, die nur von Hand zu bereinigen sind — der Einrichtungsdialog muss das verhindern.

3 · Serverschema

PostgreSQL 15 oder neuer. Alle Zeitstempel sind TIMESTAMPTZ in UTC; die App rechnet für die Anzeige um. Geldbeträge sind NUMERIC(12,2) und nicht double precision — Gleitkomma und Kassenbuch vertragen sich nicht.

3.1 Synchronisations-Grundgerüst

Jede fachliche Tabelle trägt dieselben fünf Spalten. server_seq stammt aus einer einzigen globalen Sequenz und ist der Lesezeiger für den Abgleich: Ein Client merkt sich die höchste Nummer, die er gesehen hat, und fragt beim nächsten Mal nur nach, was darüber liegt.

```
CREATE SEQUENCE sync_seq;

CREATE OR REPLACE FUNCTION touch_sync() RETURNS trigger AS $$
BEGIN
    NEW.server_seq := nextval('sync_seq');
    NEW.updated_at := now();
    RETURN NEW;
END $$ LANGUAGE plpgsql;

-- je Tabelle:
-- server_seq      BIGINT      NOT NULL
-- updated_at      TIMESTAMPTZ NOT NULL
-- deleted         BOOLEAN     NOT NULL DEFAULT false
-- deleted_at      TIMESTAMPTZ
-- origin_device   UUID        REFERENCES devices(id)
-- CREATE TRIGGER trg_sync BEFORE INSERT OR UPDATE ON <tabelle>
-- FOR EACH ROW EXECUTE FUNCTION touch_sync();
-- CREATE INDEX ON <tabelle> (server_seq);
```

Ein globaler Zähler statt einer Sequenz je Tabelle: so genügt dem Client ein einziger Lesezeiger für den gesamten Abgleich.

3.2 Geräte und Idempotenz

```
CREATE TABLE devices (
    id          UUID PRIMARY KEY,
    label       TEXT      NOT NULL,          -- "Theke links", "iPad Garten"
    platform    TEXT      NOT NULL CHECK (platform IN ('android','ios')),
    token_hash  TEXT      NOT NULL,          -- Argon2id des Geraetetokens
    last_seen_at TIMESTAMPTZ,
    last_ack_seq BIGINT    NOT NULL DEFAULT 0,
    revoked     BOOLEAN    NOT NULL DEFAULT false,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Jede Push-Operation traegt eine client_change_id. Der Server merkt sich, welche
-- er bereits angewandt hat, damit ein Wiederholungsversuch nach Netzabbruch nicht
-- dieselbe Buchung ein zweites Mal schreibt.
CREATE TABLE applied_changes (
    client_change_id UUID PRIMARY KEY,
    device_id        UUID      NOT NULL REFERENCES devices(id),
    entity           TEXT      NOT NULL,
    entity_id        UUID      NOT NULL,
    applied_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    result           TEXT      NOT NULL      -- 'applied' | 'ignored_stale'
);
CREATE INDEX ON applied_changes (device_id, applied_at);
```

Ohne applied_changes führt jeder Wiederholungsversuch zu Doppelbuchungen – genau der Fall, der bei schlechtem WLAN ständig auftritt.

3.3 Stammdaten

```
CREATE TABLE member_categories (  
  id          UUID PRIMARY KEY,  
  name        TEXT          NOT NULL,  
  negative_balance_limit NUMERIC(12,2) NOT NULL DEFAULT 0,  
  -- + Sync-Grundgeruest aus 3.1  
);  
  
CREATE TABLE members (  
  id          UUID PRIMARY KEY,  
  name        TEXT          NOT NULL,  
  category_id UUID REFERENCES member_categories(id),  
  last_used_timestamp TIMESTAMPTZ,  
  -- kein balance: abgeleitet, siehe 2.2  
);  
  
CREATE TABLE products (  
  id          UUID PRIMARY KEY,  
  name        TEXT          NOT NULL,  
  price       NUMERIC(12,2) NOT NULL,  
  category    TEXT          NOT NULL DEFAULT '',  
  image_url   TEXT,  
  has_variants BOOLEAN      NOT NULL DEFAULT false,  
  serving_size DOUBLE PRECISION NOT NULL DEFAULT 1.0  
);  
  
CREATE TABLE product_variants (  
  id          UUID PRIMARY KEY,  
  product_id  UUID          NOT NULL REFERENCES products(id),  
  name        TEXT          NOT NULL,  
  price       NUMERIC(12,2) NOT NULL,  
  serving_size DOUBLE PRECISION -- NULL erbt vom Produkt  
);  
  
CREATE TABLE stock_items (  
  id          UUID PRIMARY KEY,  
  name        TEXT NOT NULL,  
  unit        TEXT NOT NULL DEFAULT 'Stk',  
  tracking     TEXT NOT NULL CHECK (tracking IN ('SIMPLE','CONTAINER')),  
  min_level   DOUBLE PRECISION NOT NULL DEFAULT 0  
  -- kein simple_quantity: abgeleitet, siehe 2.3  
);  
  
CREATE TABLE container_types (  
  id          UUID PRIMARY KEY,  
  stock_item_id UUID NOT NULL REFERENCES stock_items(id),  
  label       TEXT NOT NULL,  
  nominal_size DOUBLE PRECISION NOT NULL,  
  initial_yield_estimate DOUBLE PRECISION NOT NULL  
  -- kein full_count: abgeleitet aus stock_entries minus Anstiche  
);  
  
CREATE TABLE product_components (  
  id          UUID PRIMARY KEY,  
  product_id  UUID NOT NULL REFERENCES products(id),  
  stock_item_id UUID NOT NULL REFERENCES stock_items(id),  
  quantity_per_unit DOUBLE PRECISION NOT NULL,  
  UNIQUE (product_id, stock_item_id)  
);
```

Mengen bleiben DOUBLE PRECISION – 0,33 Liter ist eine Messgröße, kein Geldbetrag. Preise sind durchgängig NUMERIC.

3.4 Bewegungsdaten

Diese Tabellen sind anfügend. Sie sind der Grund, warum der Abgleich überhaupt konfliktfrei sein kann: zwei Geräte schreiben verschiedene Zeilen, nie dieselbe.

```
CREATE TABLE transactions (
  id                UUID PRIMARY KEY,
  transaction_group_id UUID      NOT NULL,  -- klammert einen Kassiertvorgang
  member_id         UUID          REFERENCES members(id),
  member_name       TEXT,
  product_ref       UUID          NOT NULL,  -- Nullsentinel = Guthabenbewegung
  product_name      TEXT          NOT NULL,
  product_category  TEXT          NOT NULL DEFAULT '',
  price             NUMERIC(12,2) NOT NULL,
  quantity          INTEGER       NOT NULL,
  discount_amount   NUMERIC(12,2) NOT NULL DEFAULT 0,
  payment_type      TEXT          NOT NULL,  -- CASH|CARD|MEMBER_BALANCE|TOPUP_*|CORRECTION
  occurred_at       TIMESTAMPTZ   NOT NULL,
  is_refund         BOOLEAN       NOT NULL DEFAULT false,
  note             TEXT
);
CREATE INDEX ON transactions (member_id, occurred_at);
CREATE INDEX ON transactions (occurred_at);
CREATE INDEX ON transactions (transaction_group_id);

CREATE TABLE deliveries (
  id                UUID PRIMARY KEY,
  supplier          TEXT          NOT NULL DEFAULT '',
  receipt_total     NUMERIC(12,2),
  photo_key         TEXT,
  note             TEXT,
  occurred_at       TIMESTAMPTZ   NOT NULL
);

CREATE TABLE stock_entries (
  id                UUID PRIMARY KEY,
  stock_item_id     UUID          NOT NULL REFERENCES stock_items(id),
  item_name         TEXT          NOT NULL,  -- Schnappschuss
  quantity          DOUBLE PRECISION NOT NULL,  -- negativ = Korrektur
  unit_label        TEXT          NOT NULL,
  total_cost        NUMERIC(12,2),
  note             TEXT,
  source            TEXT          NOT NULL CHECK (source IN ('MANUAL','SCAN','CORRECTION')),
  occurred_at       TIMESTAMPTZ   NOT NULL,
  delivery_id       UUID          REFERENCES deliveries(id)
);

CREATE TABLE tapped_containers (
  id                UUID PRIMARY KEY,
  container_type_id UUID          NOT NULL REFERENCES container_types(id),
  opened_at         TIMESTAMPTZ   NOT NULL,
  closed_at         TIMESTAMPTZ,
  close_reason      TEXT CHECK (close_reason IN ('EMPTIED','SPOILED')),
  discarded_volume  DOUBLE PRECISION NOT NULL DEFAULT 0,
  note             TEXT
  -- kein drawn: abgeleitet aus transactions seit opened_at ueber die Rezeptur
);
```

photo_uri wird zu photo_key: eine Android-Content-URI ist auf dem iPad bedeutungslos. Belegfotos gehen in einen Objektspeicher, siehe 5.5.

Eine Tabelle, die es nicht gibt

Es gibt bewusst keine Tabelle für Einstellungen. Vereinsname, Vereinsfarbe, Hell/Dunkel und der SumUp-Affiliate-Key bleiben gerätelokal. Der Affiliate-Key ist ein Zugangsgeheimnis und hat in einem Sync-Payload nichts verloren; die Darstellungseinstellungen sind pro Gerät sinnvoll verschieden — das Wandtablet hinter der Theke will dunkel bleiben, auch wenn das iPad im Garten hell läuft.

4 · Synchronisationsprotokoll

Zwei Richtungen, beide vom Client angestoßen. Der Server ruft nie von sich aus an — das spart Push-Infrastruktur und funktioniert hinter jedem Heimrouter.

4.1 Ziehen

```
GET /v1/sync/changes?since=4711&limit=500
Authorization: Bearer <geraetetoken>

200 OK
{
  "changes": [
    { "entity": "products", "seq": 4712, "deleted": false,
      "row": { "id": "018f...", "name": "Weissbier 0,5l", "price": "4.20",
        "category": "Getraenke", "has_variants": false,
        "serving_size": 0.5, "updated_at": "2026-09-15T18:22:10Z" } },
    { "entity": "transactions", "seq": 4713, "deleted": false,
      "row": { "id": "018f...", "member_id": "018e...", "price": "4.20",
        "quantity": 1, "payment_type": "MEMBER_BALANCE",
        "occurred_at": "2026-09-15T18:22:09Z", ... } }
  ],
  "next_since": 4713,
  "has_more": false,
  "server_time": "2026-09-15T18:22:31Z"
}
```

Eine gemischte Liste in Sequenzreihenfolge statt eines Blocks je Tabelle: so kommen abhängige Zeilen nie vor der Zeile an, auf die sie verweisen.

Der Client schreibt den Block in einer einzigen lokalen Transaktion und speichert erst danach `next_since`. Bricht die Verbindung mitten drin ab, wird derselbe Block beim nächsten Mal wiederholt — die Zeilen sind identisch, das Ergebnis also dasselbe.

4.2 Schieben

```

POST /v1/sync/push
Authorization: Bearer <geraetetoken>

{
  "device_id": "018e-aaaa-...",
  "operations": [
    { "client_change_id": "018f-1111-...",          // Idempotenzschluessel
      "entity": "transactions",
      "op": "insert",
      "row": { "id": "018f-2222-...", "transaction_group_id": "018f-3333-...",
                "member_id": "018e-bbbb-...", "product_ref": "018e-cccc-...",
                "product_name": "Weissbier 0,5l", "price": "4.20", "quantity": 1,
                "payment_type": "MEMBER_BALANCE",
                "occurred_at": "2026-09-15T18:22:09Z" } },
    { "client_change_id": "018f-4444-...",
      "entity": "products",
      "op": "update",
      "base_updated_at": "2026-09-15T17:02:00Z", // fuer Konflikterkennung
      "row": { "id": "018e-cccc-...", "price": "4.40" } }
  ]
}

200 OK
{
  "results": [
    { "client_change_id": "018f-1111-...", "status": "applied",          "seq": 4801 },
    { "client_change_id": "018f-4444-...", "status": "ignored_stale",
      "current": { "price": "4.50", "updated_at": "2026-09-15T18:10:00Z" } }
  ],
  "next_since": 4801
}

```

Die Operationen eines Aufrufs werden serverseitig in einer Transaktion angewandt: entweder alle oder keine.

4.3 Konfliktregeln je Tabelle

Tabelle	Art	Regel bei gleichzeitiger Änderung
transactions	anfügend	Kein Konflikt möglich. Doppelte werden über client_change_id abgefangen.
stock_entries	anfügend	Wie oben.
deliveries	anfügend	Wie oben.
tapped_containers	Ereignis	Anstich und Fasswechsel sind Ereignisse. Stechen zwei Geräte dasselbe Gebinde an, gewinnt der frühere opened_at; das zweite wird verworfen und dem Gerät gemeldet.
products, variants, components	Stammdaten	Letzter Schreibvorgang gewinnt, verglichen über base_updated_at. Ist die Serverzeile neuer, wird die Operation als ignored_stale zurückgemeldet und die App zeigt den Serverwert.
members, member_categories	Stammdaten	Wie Produkte. Der Saldo ist davon nicht betroffen, weil er kein Feld mehr ist.
stock_items, container_types	Stammdaten	Wie Produkte.
member_balances, Bestandswerte	abgeleitet	Nur lesbar. Ein Client, der versucht sie zu schreiben, bekommt 422.

4.4 Wann abgeglichen wird

- Nach jedem abgeschlossenen Kassiovorgang, sofort, mit kurzem Timeout.
- Alle 60 Sekunden im Hintergrund, solange die App im Vordergrund ist.
- Beim Wechsel in den Vordergrund und bei wiedererlangter Netzverbindung.
- Bei offener Warteschlange mit Backoff: 2s, 4s, 8s, dann jede Minute.

Sichtbarkeit ist Teil der Anforderung

Die App muss erkennbar machen, wann sie zuletzt abgeglichen hat und wie viele Buchungen noch in der Warteschlange stehen. Ein Kassier, der nicht weiß, dass sein Gerät seit einer Stunde allein arbeitet, trifft falsche Entscheidungen — etwa, einen Deckel zu belasten, der längst am Limit ist.

5 · Schnittstellenreferenz

5.1 Endpunkte

Methode und Pfad	Zweck
GET /v1/health	Erreichbarkeitsprüfung. Wird vom Einrichtungsdiallog benutzt, um eine eingegebene API-URL zu validieren. Ohne Token.
POST /v1/devices/register	Geräteanmeldung gegen den Kopplungscode. Liefert Geräte-ID und Token.
GET /v1/sync/changes	Ziehen ab Lesezeiger.
POST /v1/sync/push	Schieben mit Idempotenzschlüsseln.
POST /v1/media/receipts	Belegfoto hochladen, liefert photo_key.
GET /v1/media/receipts/{key}	Belegfoto abrufen.
GET /v1/members/{id}/balance	Einzelsaldo, für die Prüfung unmittelbar vor einer Deckelbelastung.

5.2 Anmeldung eines Geräts

Kein Passwort in der App. Der Administrator erzeugt am Server einen Kopplungscode mit kurzer Laufzeit, der auf dem Gerät einmalig eingegeben wird. Das Gerät erhält daraufhin ein eigenes, dauerhaftes Token, das im Schlüsselbund des Betriebssystems liegt — Keystore unter Android, Keychain unter iOS.

```
POST /v1/devices/register
{ "pairing_code": "8K4M-2QX9", "label": "iPad Garten", "platform": "ios" }

201 Created
{ "device_id": "018e-aaaa-...", "token": "vd_dev_9c1f...", "initial_since": 0 }
```

Das Token wird serverseitig nur als Argon2id-Hash abgelegt. Geht ein Gerät verloren, wird es einzeln gesperrt, ohne die anderen anzufassen.

5.3 Fehlerbilder

Code	Bedeutung	Verhalten der App
401	Token ungültig oder gesperrt	Abgleich stoppen, Neukopplung anbieten. Lokaler Betrieb läuft weiter.
409	Kopplungscode verbraucht	Meldung im Einrichtungsdiallog.
422	Schreibversuch auf abgeleitetes Feld	Fehler protokollieren. Das ist ein Programmfehler, kein Betriebsfall.
429	Zu viele Anfragen	Backoff nach 4.4.
5xx / Timeout	Server nicht erreichbar	Warteschlange behalten, später erneut. Kein Datenverlust, keine Fehlermeldung im Verkaufsweg.

5.4 Zahlenformate

- Geld wird als **Zeichenkette** übertragen ("4.20"), nicht als JSON-Zahl. JSON-Zahlen sind Gleitkomma, und 4,20 ist dort nicht exakt darstellbar.

- Mengen sind JSON-Zahlen — sie sind Messwerte, kleine Abweichungen sind belanglos.
- Zeitstempel sind ISO 8601 in UTC mit Z.
- UUIDs in Kleinschreibung mit Bindestrichen.

5.5 Belegfotos

Der Wareneingang hält ein Foto des Kassabons. Heute steht dort eine Android-URI, die auf keinem anderen Gerät etwas bedeutet. Künftig wird das Bild beim Abgleich hochgeladen; die Zeile speichert nur den zurückgegebenen Schlüssel. Bis der Upload durch ist, bleibt lokal eine Kopie liegen, damit der Beleg auch offline sichtbar ist.

6 · Kartenzahlung im Mehrgerätebetrieb

SumUp ist gerätegebunden: Das Terminal koppelt sich über Bluetooth mit genau einem Tablet, und das jeweilige SDK — Android beziehungsweise iOS — wickelt die Zahlung lokal ab. Der Server ist daran nicht beteiligt und soll es auch nicht sein.

- Der Affiliate-Key bleibt gerätelokal und wird nicht synchronisiert.
- Jedes Gerät meldet sich einmal bei SumUp an; die Anmeldung ist nicht übertragbar.
- Ein Kartenterminal ist zu einem Zeitpunkt mit einem Gerät gekoppelt. Zwei Kassen brauchen zwei Terminals.
- Gelingt die Zahlung, entsteht eine gewöhnliche Transaktionszeile mit `payment_type = 'CARD'`, die wie jede andere abgeglichen wird.

Was auf iOS anders ist

Die beiden SumUp-SDKs sind nicht dasselbe Produkt mit zwei Hüllen, sondern zwei getrennte Bibliotheken mit unterschiedlichen Abläufen. Die Android-Seite arbeitet über einen Activity-Result-Vertrag, die iOS-Seite über Delegates und View-Controller-Präsentation. Im geteilten Modul steht deshalb nur eine schmale Schnittstelle, deren beide Umsetzungen jeweils in ihrer eigenen Welt bleiben. Verifizieren lässt sich die iOS-Seite ausschließlich auf einem Mac mit echtem Terminal — ein Simulator kann keine Bluetooth-Kartenzahlung.

7 · Betrieb

7.1 Aufstellung

```
# Minimal, und fuer einen Verein voellig ausreichend:
# ein kleiner Server (auch ein Raspberry Pi 5 genuegt), Docker, Reverse Proxy
services:
  db:
    image: postgres:16
    environment: [POSTGRES_DB=vereinsdeckel, POSTGRES_PASSWORD=...]
    volumes: ["/pgdata:/var/lib/postgresql/data"]
  api:
    image: vereinsdeckel/api:1.0
    environment: [DATABASE_URL=postgres://..., PAIRING_ADMIN_TOKEN=...]
    depends_on: [db]
  proxy:
    image: caddy:2          # TLS-Zertifikat automatisch ueber Let's Encrypt
    ports: ["443:443"]
```

Die App verlangt HTTPS. Ein selbstsigniertes Zertifikat wird von iOS abgelehnt, solange es nicht als Profil installiert ist — der einfachere Weg ist ein echter Hostname mit Let's-Encrypt-Zertifikat.

7.2 Sicherung

- Nächtlicher `pg_dump`, sieben Tagesstände plus vier Wochenstände.
- Belegfotos liegen außerhalb der Datenbank und brauchen eine eigene Sicherung.
- Die vorhandene Backupfunktion der App bleibt bestehen: Sie sichert den lokalen Stand des Geräts und ist die Absicherung für den Fall, dass der Server abraucht, bevor etwas abgeglichen wurde.
- Eine Rücksicherung muss einmal geübt worden sein. Ein ungeprüftes Backup ist eine Vermutung.

7.3 Datenschutz

Mitgliedsnamen mit Kontostand sind personenbezogene Daten. Mit dem Schritt vom Tablet auf einen Server wird daraus eine Verarbeitung, die der Verein dokumentieren muss: Verzeichnis der Verarbeitungstätigkeiten, Zweckbindung, Löschfristen für ausgetretene Mitglieder, und ein Auftragsverarbeitungsvertrag, falls der Server nicht beim Verein selbst steht. Das ist kein technisches Detail, sondern eine Voraussetzung für den Produktivbetrieb.

8 · Abnahmekriterien

Woran sich prüfen lässt, ob der Mehrgerätebetrieb wirklich funktioniert. Jeder Punkt ist von Hand mit zwei Geräten nachstellbar.

Nr.	Szenario	Erwartetes Ergebnis
A1	Beide Geräte online, auf Gerät A ein Produkt anlegen	Erscheint auf B binnen 60 Sekunden, spätestens nach dem nächsten Kassivorgang.
A2	Gerät B im Flugmodus, drei Verkäufe buchen, Flugmodus aus	Alle drei erscheinen auf A. Keine Dublette, keine verlorene Zeile.
A3	Deckel mit 20 EUR, gleichzeitig 4 EUR auf A und 5 EUR auf B buchen	Nach dem Abgleich zeigen beide Geräte 11 EUR. Das ist der Test, der ohne die Änderung aus 2.2 fehlschlägt.
A4	Push-Antwort durch Netzabbruch verloren, Client wiederholt	Die Buchung steht genau einmal in der Historie.
A5	Auf beiden Geräten denselben Produktpreis ändern	Der spätere Schreibvorgang gewinnt, das andere Gerät übernimmt den Serverwert sichtbar.
A6	Fass auf A anstecken, während B offline war und es ebenfalls ansteckt	Ein Anstich bleibt bestehen, der andere wird verworfen und gemeldet.
A7	Server abschalten, weiter kassieren	Beide Geräte verkaufen unverändert weiter. Die Warteschlange ist sichtbar.
A8	Server wieder anschalten	Beide Warteschlangen laufen leer, Bestände und Salden stimmen überein.
A9	Kartenzahlung auf dem iPad	Zahlung geht über das gekoppelte Terminal, die Zeile erscheint auf beiden Geräten.
A10	Belegfoto auf A erfassen	Foto ist auf B sichtbar, ohne dass A noch erreichbar sein muss.

Die Reihenfolge, in der gebaut werden sollte

Zuerst Schema und die drei Modelländerungen, dann der Abgleich für die anfügenden Tabellen, danach die Stammdaten, zuletzt Medien. Der Grund: Die anfügenden Tabellen sind der Teil, der Geld führt, und zugleich der konfliktfreie — er lässt sich früh vollständig richtig bekommen. Stammdatenkonflikte sind ärgerlich, aber sie kosten keine Einnahmen.